FIT1058 — Chapter 4: Propositional Logic

Exercise Solutions

Alston

Semester 2, 2026

Exercise 8

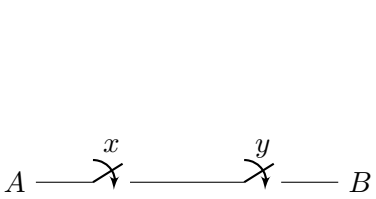
This question is about using Boolean algebra to describe the algebra of switching.

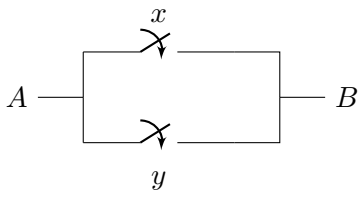
An electrical switch is represented diagrammatically as follows.

The state of a switch can be represented by a Boolean variable, with the states Off and On being represented by *False* and *True* respectively. Let x be a Boolean variable representing the state of a switch. Then $x = \mathit{True}$ represents the switch being On, so that electrical current can pass through it; $x = \mathit{False}$ represents the switch being Off, so that there is no electrical current through the switch.

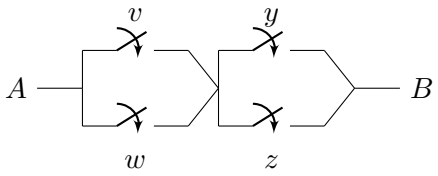
We can put switches together to make more complicated circuits. Those circuits can then be described by Boolean expressions in the variables that represent the switches. In the following circuits, v, w, x, y, z are Boolean variables representing the indicated switches.

(a) For each of the two switching circuits below, write a Boolean expression in x and y for the proposition that current flows between the two ends of the circuit. (The ends are shown as A and B in the diagram. In effect, the diagrams only show part of a complete circuit. The rest of the circuit would contain a power supply, such as a battery, and an electrical device that operates when the current flows, such as a light or an appliance.)

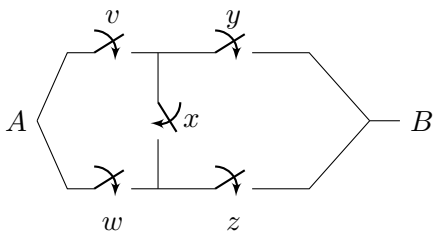




(b) For each of the next two circuits, again construct a Boolean expression to represent the proposition that current flows between the two ends of the circuit. Compare the two expressions you construct. What can you say about the relationship between them?



(c) Similarly, construct Boolean expressions for the next two circuits. Compare the two expressions you construct. What can you say about the relationship between them?



(d) For each of our circuit pairs in (a)–(c), discuss how they are related to each other, and what this means for how the Boolean expressions derived from them are related to each other.

Solution. **(a)** Switches x, y in series (current must pass through both):

$$x \wedge y$$

Switches x, y in parallel (current can pass through either):

$$x \vee y$$

(b) Left circuit: v–y in series form one branch, w–z in series form another branch, and the two branches are in parallel:

$$(v \wedge y) \vee (w \wedge z)$$

Right circuit: v–w in parallel form one stage, y–z in parallel form another stage, and the two stages are in series:

$$(v \vee y) \wedge (w \vee z)$$

Comparing the two expressions: swapping $\wedge \leftrightarrow \vee$ throughout one expression gives the other. The two circuits are *duals* of each other, and the two Boolean expressions are dual expressions.

(c) This is a bridge (Wheatstone) circuit, with x as the bridging switch. We compute by case analysis on x.

When x is False (the bridge is open, so the circuit reduces to the non-crossed form from (b)):

$$\neg x \wedge \big((v \wedge y) \vee (w \wedge z)\big)$$

When x is True (the bridge is closed, shorting the two middle nodes together, so v, w become parallel into the shared node, and y, z become parallel out of it, in series with each other):

$$x \wedge (v \vee w) \wedge (y \vee z)$$

Combining the two cases:

$$\big(x \wedge (v \vee w) \wedge (y \vee z)\big) \vee \big(\neg x \wedge ((v \wedge y) \vee (w \wedge z))\big)$$

Simplification. Expanding $x \wedge (v \vee w) \wedge (y \vee z)$ by distributivity:

$$x \wedge (v \vee w) \wedge (y \vee z) \equiv (x \wedge v \wedge y) \vee (x \wedge v \wedge z) \vee (x \wedge w \wedge y) \vee (x \wedge w \wedge z)$$

and the other branch is already

$$\neg x \wedge ((v \wedge y) \vee (w \wedge z)) \equiv (\neg x \wedge v \wedge y) \vee (\neg x \wedge w \wedge z).$$

Collecting all six terms:

$$(x \wedge v \wedge y) \vee (x \wedge v \wedge z) \vee (x \wedge w \wedge y) \vee (x \wedge w \wedge z) \vee (\neg x \wedge v \wedge y) \vee (\neg x \wedge w \wedge z)$$

The pairs $(x \wedge v \wedge y) \vee (\neg x \wedge v \wedge y)$ and $(x \wedge w \wedge z) \vee (\neg x \wedge w \wedge z)$ each factor out, using $x \vee \neg x \equiv \mathsf{True}$:

$$(x \wedge v \wedge y) \vee (\neg x \wedge v \wedge y) \equiv (v \wedge y) \wedge (x \vee \neg x) \equiv v \wedge y,$$

$$(x \wedge w \wedge z) \vee (\neg x \wedge w \wedge z) \equiv (w \wedge z) \wedge (x \vee \neg x) \equiv w \wedge z.$$

The remaining two terms, $(x \wedge v \wedge z)$ and $(x \wedge w \wedge y)$, do not simplify further. Hence:

$$(v \wedge y) \vee (w \wedge z) \vee (x \wedge v \wedge z) \vee (x \wedge w \wedge y)$$

■

Exercise 19

Prove, by induction on n, that the conjunction of 2^n inputs can be computed in time nt by a suitable combination of AND gates.

Hence prove that the conjunction of m arguments (where m is not necessarily a power of 2) can be computed by a suitable combination of AND gates in time $\lceil \log_2 m \rceil$.

Proof. **Part 1: Induction on n.**

Base case ($n = 1$). With two inputs x_1, x_2, a single AND gate computes $x_1 \wedge x_2$ in time $t = 1 \cdot t$. So the claim holds for $n = 1$.

Inductive step. Assume the conjunction of 2^n inputs can be computed in time nt by a suitable combination of AND gates.

Now suppose we have $2^{n+1} = 2 \cdot 2^n$ inputs $x_1, \ldots, x_{2^{n+1}}$. Split them into two blocks of 2^n inputs each:

$$(x_1 \wedge \cdots \wedge x_{2^n}) \quad \text{and} \quad (x_{2^n+1} \wedge \cdots \wedge x_{2^{n+1}}).$$

By the inductive hypothesis, each block can be computed in time nt (in parallel, since the two blocks use disjoint inputs). After time nt, we have exactly two values available, and a single additional AND gate combines them, taking a further time t.

Hence the total time is

$$nt + t = (n + 1)t.$$

By induction, the conjunction of 2^n inputs can be computed in time nt for all $n \geq 1$.

Part 2: General m.

Case 1: $m = 2^n$ for some n. Then $\log_2 m = n$, so $\lceil \log_2 m \rceil = n$, and by Part 1 the total time is $nt = \lceil \log_2 m \rceil \, t$, as required.

Case 2: m is not a power of 2. Let

$$k = \lceil \log_2 m \rceil, \qquad M = 2^k.$$

Since $k \geq \log_2 m$ and $x \mapsto 2^x$ is increasing,

$$2^k \geq 2^{\log_2 m} = m, \qquad \text{i.e.} \qquad M \geq m.$$

We have m real inputs $x_1, \ldots, x_m$, with $m < M$. Pad the input list with $M - m$ copies of the constant True:

$$x_1 \wedge \cdots \wedge x_m \wedge \underbrace{\mathsf{True} \wedge \cdots \wedge \mathsf{True}}_{M-m}.$$

Since $P \wedge \mathsf{True} \equiv P$ for any proposition P, this padded conjunction is logically equivalent to the original one:

$$x_1 \wedge \cdots \wedge x_m \wedge \underbrace{\mathsf{True} \wedge \cdots \wedge \mathsf{True}}_{M-m} \;\equiv\; x_1 \wedge \cdots \wedge x_m.$$

The padded expression has exactly $M = 2^k$ inputs, so by Part 1 it can be computed by a suitable combination of AND gates in time

$$kt = \lceil \log_2 m \rceil \, t.$$

Since the padded expression is equivalent to $x_1 \wedge \cdots \wedge x_m$, the same circuit computes $x_1 \wedge \cdots \wedge x_m$ in time $\lceil \log_2 m \rceil \, t$.

Combining both cases, for every $m \geq 1$ the conjunction of m arguments can be computed by a suitable combination of AND gates in time $\lceil \log_2 m \rceil \, t$. ■

Exercise 20

A Boolean expression is called *affine* if it is of one of the following forms:

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n \qquad \text{or} \qquad x_1 \oplus x_2 \oplus \cdots \oplus x_n \oplus \mathit{True}.$$

Prove, by induction on n, that for all $n \in \mathbb{N}$, the number of satisfying truth assignments of an affine Boolean expression with n variables is 2^{n-1}.

It follows that half of all truth assignments are satisfying for the expression, and half are not.

Here, a *truth assignment* is just an assignment of a truth value to each variable, and it is *satisfying* if the assignment makes the whole expression *True*.

Proof. **Base case ($n = 2$).** Consider $x_1 \oplus x_2$:

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Exactly 2 of the 4 assignments satisfy the expression, and $2^{2-1} = 2$. So the base case holds. (The case with the extra $\oplus$ True simply flips every output, which does not change the *count* of satisfying assignments, by the same argument as below.)

Inductive hypothesis. Assume that for some n, any affine Boolean expression with n variables has exactly 2^{n-1} satisfying truth assignments.

Inductive step. Suppose we have $n + 1$ variables, and let

$$Q = x_1 \oplus x_2 \oplus \cdots \oplus x_n \quad (\text{or } x_1 \oplus \cdots \oplus x_n \oplus \text{True}),$$

so that Q is itself an affine expression on n variables. The $(n+1)$-variable affine expression is then

$$Q \oplus x_{n+1}.$$

By definition of $\oplus$, $Q \oplus x_{n+1}$ is True exactly when Q and x_{n+1} differ:

Q	x_{n+1}	$Q \oplus x_{n+1}$
0	0	0
0	1	1
1	0	1
1	1	0

So $Q \oplus x_{n+1}$ is satisfied exactly in the following two disjoint cases:

Case 1: Q = False, x_{n+1} = True. By the inductive hypothesis, Q is True for 2^{n-1} of the 2^n assignments to $x_1, \ldots, x_n$, so Q is False for the remaining $2^n - 2^{n-1}$ assignments. Combined with x_{n+1} = True (1 way), this gives

$$2^n - 2^{n-1}$$

satisfying assignments.

Case 2: Q = True, x_{n+1} = False. By the inductive hypothesis, Q is True for 2^{n-1} assignments to $x_1, \ldots, x_n$. Combined with x_{n+1} = False (1 way), this gives

$$2^{n-1}$$

satisfying assignments.

Since these two cases are disjoint (they differ in the value of x_{n+1}), the total number of satisfying assignments for the $(n+1)$-variable expression is

$$(2^n - 2^{n-1}) + 2^{n-1} = 2^n = 2^{(n+1)-1}.$$

This matches the claimed formula for $n+1$ variables.

Conclusion. By induction, for all $n \in \mathbb{N}$, an affine Boolean expression with n variables has exactly 2^{n-1} satisfying truth assignments. Since there are 2^n truth assignments in total, exactly half are satisfying and half are not. ■

Exercise 21

Let x_1x_0 be the bits of a two-bit binary number x, representing an integer in $\{0,1,2,3\}$. So $x_1, x_0 \in \{0,1\}$ and we have $x = 2x_1 + x_0$.

Let $y_2y_1y_0$ be the three-bit binary representation of the number $x+1$ obtained from x by incrementing it. So $y \in \{1,2,3,4\}$, and its leading bit is 0 except if $x = 3$, in which case $y = 4$, which in binary is 100.

In this exercise we use 0 and 1 for the truth values *False* and *True* (as mentioned on p. 122 in § 4.1α).

Give Boolean expressions for each of the three bits of y, in terms of the two bits x_1 and x_0 of x.

Solution. **Truth table.**

x_1	x_0	x	$x+1$	y_2	y_1	y_0
0	0	0	1	0	0	1
0	1	1	2	0	1	0
1	0	2	3	0	1	1
1	1	3	4	1	0	0

Bit y_2. This bit is 1 only in the last row, where $x_1 = x_0 = 1$:

$$y_2 \equiv x_1 \wedge x_0$$

Bit y_1. This bit is 1 exactly when x_1, x_0 differ:

$$y_1 \equiv (\neg x_1 \wedge x_0) \vee (x_1 \wedge \neg x_0) \equiv x_1 \oplus x_0$$

Bit y_0. From the table:

$$y_0 \equiv (\neg x_1 \wedge \neg x_0) \vee (x_1 \wedge \neg x_0)$$

Factor out $\neg x_0$:

$$\equiv \neg x_0 \wedge (\neg x_1 \vee x_1)$$

Since $\neg x_1 \vee x_1 \equiv \mathsf{True}$:

$$\equiv \neg x_0 \wedge \mathsf{True} \equiv \neg x_0$$

This matches intuition: incrementing always flips the lowest bit.

Summary.

$$y_2 \equiv x_1 \wedge x_0, \qquad y_1 \equiv x_1 \oplus x_0, \qquad y_0 \equiv \neg x_0$$

■

Exercise 22

Is the operation set $\{\oplus, \neg\}$ universal? Justify your answer.

Solution. By Exercise 20, any affine expression (i.e. any expression built from $\oplus, \neg$, using $\neg P \equiv P \oplus \text{True}$) in n variables has exactly 2^{n-1} satisfying assignments.

x_1	x_2	$x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

$x_1 \vee x_2$ has 3 satisfying assignments; $x_1 \wedge x_2$ has 1. Neither equals $2^{2-1} = 2$, so $\wedge, \vee$ cannot be expressed correctly by $\{\oplus, \neg\}$.

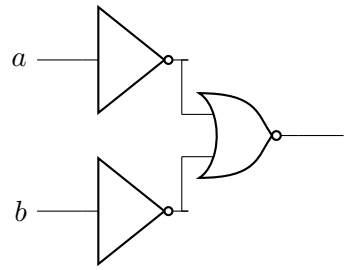
Hence $\{\oplus, \neg\}$ is **not** a universal set of operations. ■

Exercise 24

Computer circuits actually perform the very same Boolean logic that we have studied using *logic gates* (§ 4.4α). The idea is that you have one or more wires coming into a gate, and usually one output wire. If an input wire has current flowing through, the variable it represents is set to *True*, and if not, then *False*. The logic gate then takes those signals and gives off an output signal if the result should be *True*, and no signal if the result is *False*. Below are the most common logic gates seen in circuits.

The AND, OR, and NOT gates function the same as what we have seen previously. If both signals of an AND gate are on, it will output a signal. For OR, only one input signal is required to be on, and for NOT, there will only be an output signal if the input signal is off (*False*).

XOR is "EXCLUSIVE OR", which outputs a signal only if a or b are on, but not both. NAND, NOR, and XNOR are simply the inversions of AND, OR, and XOR respectively (i.e. NAND is "not both", NOR is "neither", and XNOR is "both or neither"). These gates can also be connected in series, like in the example below, which is equivalent to the logical expression $(\neg a \vee \neg b)$.



(a) What single logic gate is the above circuit equivalent to?

(b) We saw in § 4.20, that $\{\vee, \wedge, \neg\}$ is a universal set of operations, meaning that any logical expression can be expressed with only these operations. How would you express the function of an XOR gate as a Boolean expression using only these operations?

(c) Draw a circuit diagram which performs the same functionality as an XOR gate using only AND, OR, and NOT gates.

(d) Do we even need three types of gates? Given that $\{\wedge, \vee, \neg\}$ is a universal set of operations, use De Morgan's Law to prove that $\{\wedge, \neg\}$ and $\{\vee, \neg\}$ are also universal sets of operations. (Hint: Can you express $\vee$ with the other two operators?)

(e) Draw a circuit diagram that performs an OR operation using only AND and NOT gates, and also one which performs AND using only OR and NOT gates.

(f) Can you perform the function of a NOT gate, or an AND gate, with only NAND gates? What does this tell you about NAND gates?

Solution. **(a)** By De Morgan's law,

$$\neg a \vee \neg b \equiv \neg(a \wedge b) \equiv a \text{ NAND } b.$$

So the circuit (two NOT gates feeding a NOR gate) is equivalent to a single **NAND gate**.

(b) XOR outputs true exactly when a, b differ:

$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b).$$

(d) We show $\vee$ can be expressed using only $\wedge, \neg$, and $\wedge$ can be expressed using only $\vee, \neg$; since $\{\wedge, \vee, \neg\}$ is universal, this shows $\{\wedge, \neg\}$ and $\{\vee, \neg\}$ are each universal.

Using double negation ($\neg\neg P \equiv P$) and De Morgan's law:

$$a \vee b \equiv \neg\neg(a \vee b) \equiv \neg(\neg a \wedge \neg b),$$

which expresses $\vee$ using only $\wedge$ and $\neg$. Hence $\{\wedge, \neg\}$ is a universal set of operations.

$$a \wedge b \equiv \neg\neg(a \wedge b) \equiv \neg(\neg a \vee \neg b),$$

which expresses $\wedge$ using only $\vee$ and $\neg$. Hence $\{\vee, \neg\}$ is a universal set of operations.

(f) We show every operation in $\{\wedge, \vee, \neg\}$ can be built from NAND alone.

NOT:

$$\neg a \equiv \neg a \vee \neg a \equiv \neg(a \wedge a) \equiv a \text{ NAND } a.$$

OR:

$$a \vee b \equiv \neg\neg(a \vee b) \equiv \neg(\neg a \wedge \neg b) \equiv \neg a \text{ NAND } \neg b \equiv (a \text{ NAND } a) \text{ NAND } (b \text{ NAND } b).$$

AND:

$$a \wedge b \equiv \neg\big(\neg(a \wedge b)\big) \equiv \neg(a \text{ NAND } b) \equiv (a \text{ NAND } b) \text{ NAND } (a \text{ NAND } b).$$

Since NAND alone can express $\wedge, \vee, \neg$, and $\{\wedge, \vee, \neg\}$ is universal, the single operation NAND is itself a **universal (functionally complete) operation**: every Boolean circuit can, in principle, be built using nothing but NAND gates. ■